AI Mirage: The Dungeon Master

October 10, 2025

Abstract

This report details the design and implementation of "AI Mirage", an advanced text-based role-playing game powered by Large Language Models (LLMs). The project addresses fundamental challenges in LLM-driven narrative generation, specifically focusing on achieving long-term memory and narrative consistency. We present a modular architecture incorporating a hybrid memory system (Summarized Working Memory and RAG-powered Persistent Memory), a multi-provider LLM routing strategy, and dynamic NPC/Quest tracking. The report outlines the system's architecture, key features, and automated validation suite, demonstrating a robust framework for interactive AI storytelling.

1 Introduction and Problem Statement

Large Language Models (LLMs) offer unprecedented capabilities for interactive narrative generation but inherently suffer from context window limitations and a lack of persistent memory. This leads to common issues in AI-driven games: forgotten plot points, inconsistent character behavior, and a loss of player immersion over time.

The "AI Mirage" project was developed to solve this problem by creating a sophisticated, memory-augmented AI Dungeon Master. The goal is to provide a consistent, dynamic, and engaging role-playing experience where player choices have lasting consequences, supported by a robust technical architecture.

2 System Architecture

The system employs a modular architecture to separate concerns between game logic, LLM interaction, and memory management. The central 'GameLoop' orchestrates a series of specialized managers, as shown in Figure **??**.

Key Modules:

- **Game Loop:** The central orchestrator that manages turn progression and calls other modules.
- **Memory Manager:** A unified interface to the hybrid memory system.
- **LLM Clients:** An abstraction layer for communicating with different LLM providers (Groq, Google).
- **Turn Analyzer:** A specialist module that uses fast LLMs to extract structured data (events, NPCs, quests) from the narrative text after each turn.

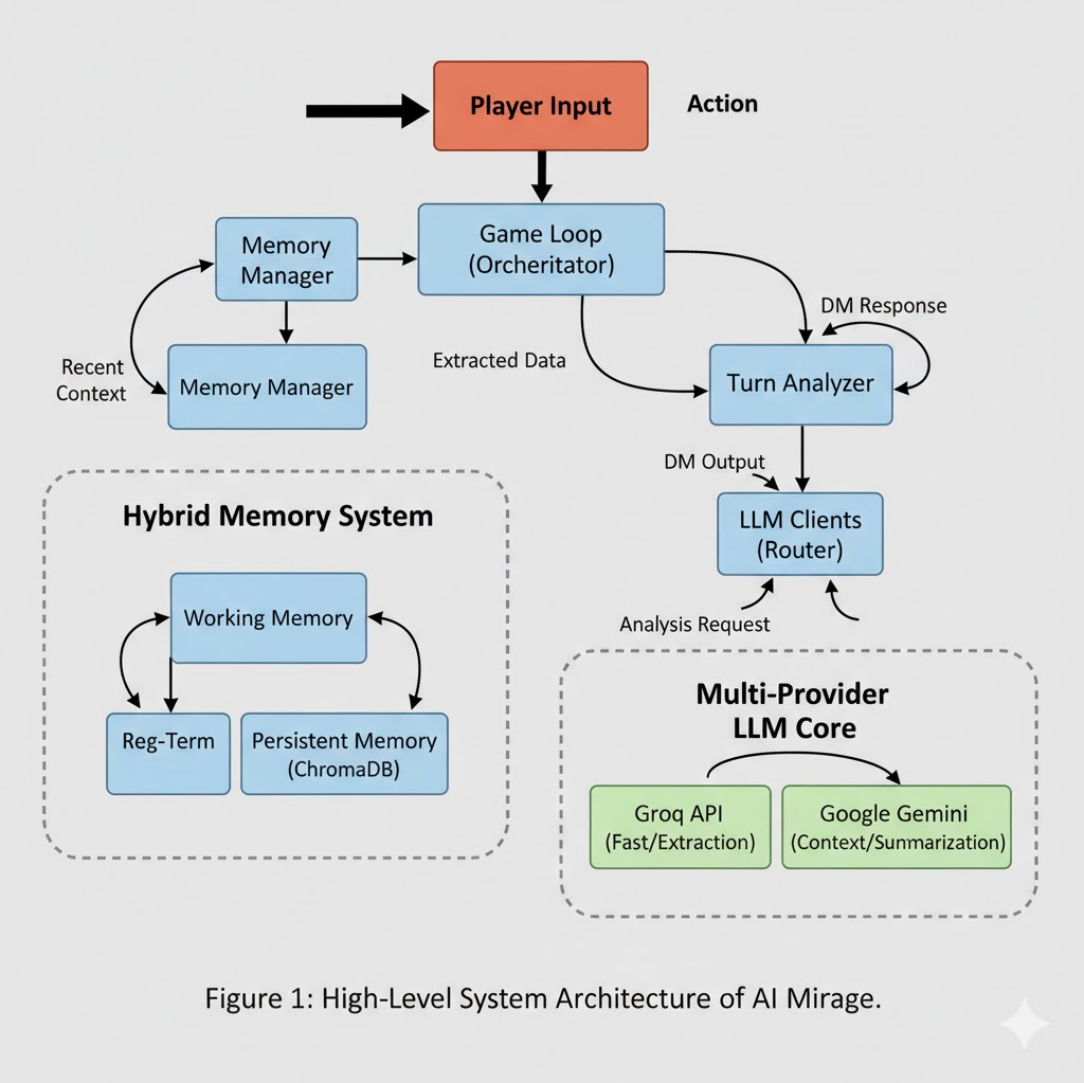


Figure 1: High-Level System Architecture of AI Mirage.

3 Key Features & Implementation

3.1 Hybrid Memory System

The core of the solution is a dual-memory architecture that provides both short-term conversational context and long-term event recall.

- **Working Memory (Short-Term):** This component holds the last ≈5 turns of raw conversation. To save tokens in the main prompt, it uses a fast LLM (e.g., 'meta-llama/llama-4-scout-17b-16e-instruct') to generate a concise, real-time summary of these recent events. This ensures conversational flow and immediate context awareness.
- **Persistent Memory (Long-Term):** This is a Retrieval-Augmented Generation (RAG) system built with **ChromaDB**. Significant events and facts extracted by the 'TurnAnalyzer' are converted into vector embeddings and stored. Before generating a new DM response, a semantic search is performed against this vector store to retrieve relevant historical events, no matter how many turns have passed. This provides the system with a true long-term memory.

3.2 Multi-Provider LLM Strategy

To optimize for performance and quality, tasks are routed to the most appropriate LLM provider:

- **Groq API:** Leveraged for its extreme speed and high rate limits. It handles all latency-sensitive tasks: main DM response generation, safety checks, and all structured data extractions.
- **Google Gemini API:** Used for specialized, high-quality background tasks. The powerful 'gemini-2.5-pro' model, with its strong reasoning, is used for the infrequent but critical recursive summarization of the entire game's history.

3.3 Dynamic NPC and Quest Memory (Bonus Features)

The system extends its memory capabilities to track specific game entities:

- **NPC Memory:** Interactions involving NPCs are stored in a separate ChromaDB collection. The system can identify which NPC is being spoken to and retrieve specific past interactions with that character, allowing for consistent and evolving relationships.
- **Quest Log:** The 'TurnAnalyzer' parses DM responses to automatically identify new quests and completions, which are tracked in a dynamic quest log. This log is fed back into the main prompt, keeping the AI aware of the player's objectives.

4 Automated QA Suite

To validate the system's functionality, an 'automated tester.py' script was developed. This "Test Conductor" uses an LLM to automatically generate player actions to create and then verify game facts over short-term (≈5 turns) and long-term (≈30 turns) scenarios. A separate LLM call evaluates whether the AI DM successfully recalled the fact, providing a 'pass/fail' status and reasoning, which is saved to 'test results.log'. This ensures the memory systems are working as designed.

5 Results & Conclusion

The implementation of the hybrid memory system and multi-provider LLM architecture successfully addresses the core problem of narrative inconsistency in LLM-driven games. As demonstrated by the logs in the 'test results/' directory, the system is capable of:

- Maintaining short-term conversational context.
- Recalling specific facts and events from over 20 turns prior.
- Tracking NPC interactions and quest statuses.

The "AI Mirage" project serves as a robust proof-of-concept that with a thoughtful memory architecture, it is possible to create persistent and immersive narrative experiences with today's LLMs. Future work could focus on tracking world-state changes and developing more complex, multi-agent NPC interactions.